Université Caen Normandie

Unité d'enseignement : Projet 2

Année universitaire 2025/2026

Détection Automatique du Plagiat Textuel

Rapport de Projet

Nom & Prénom	Rôle principal
REZGUI Léna	k-NN, BoW, BoW n-grams
Mohamed Chaib Setti	SVM, TF-IDF, TF-IDF n-grams
Aris Bensadi	MLP from scratch, générateur Blob
LAMAIRI Mohamed Yassine	Naive Bayes, NER, résultats

Corpus	PAN Plagiarism Corpus
Tâche	Classification 4 classes : `non`, `light`, `heavy`, `cut`
Classifieurs	SVM, k-NN, Naive Bayes, MLP (from scratch)
Représentations	BoW, TF-IDF, N-grammes, NER, Blob synthétique

Table des matières

1 Introduction **2**
1.1 Description générale du projet 2
1.2 Présentation du plan du rapport 2

2 Objectifs du projet **2**
2.1 Problématique 2
2.2 Points-clés et grandes étapes 3
2.3 Méthodologie et défis liés aux données 3

3 Fonctionnalités implémentées **4**
3.1 Description des fonctionnalités 4
3.2 Organisation du projet — Qui a fait quoi 6

4 Éléments techniques **6**
4.1 Méthodes de représentation textuelle 6
4.1.1 Bag of Words (BoW) et N-grammes (`bow.py`, `bow_ngrams.py`) 6
4.1.2 TF-IDF et N-grammes (`tf_idf.py`, `tf_idf_ngram.py`) 7
4.1.3 Reconnaissance d'Entités Nommées (`ner.py`) 7
4.1.4 Générateur de données synthétiques Blob (`blob.py`) 7
4.2 Algorithmes de classification 7
4.2.1 SVM avec noyau cosinus précalculé (`svm_.py`) 7
4.2.2 Réseau de neurones MLP from scratch (`mlp.py`) 8
4.2.3 Naive Bayes Complémentaire (`naivebayes.py`) 8
4.2.4 k-Plus Proches Voisins (`knn.py`) 8
4.3 Structures de données non triviales 8
4.3.1 Matrice de Gram pour le SVM 8
4.3.2 Split de référence partagé 9
4.3.3 Classe Layer et accumulation des gradients 9
4.4 Description des données 9
4.4.1 Corpus PAN 9
4.4.2 Format des CSV de représentation 9

5 Architecture du projet **10**
5.1 Paquetages non standards utilisés 10
5.2 Diagramme des modules 10
5.3 Chaînes de traitement 10
5.3.1 Phase 1 — De texte brut à vecteur numérique 10
5.3.2 Phase 2 — Classification et évaluation 11
5.3.3 Phase 3 — Visualisation des résultats 11

6 Expérimentations **11**

6.1 Exemple d'utilisation — Exécution du pipeline 11
6.2 Résultats quantifiables 12
6.2.1 Résultats SVM 12
6.2.2 Résultats k-NN 13
6.2.3 Résultats Naive Bayes 14
6.2.4 Résultats globaux — Toutes configurations confondues 15
6.2.5 Comparaison globale par classifieur 15
6.3 Analyse des résultats 15
6.3.1 TF-IDF N-grammes : la représentation dominante 15
6.3.2 Supériorité du SVM 16
6.3.3 Criticité de la régularisation SVM 16
6.3.4 Le k-NN préfère $k = 1$ 16
6.3.5 Difficulté de la classe `light` 16
6.3.6 Limitation du MLP 16

7 Conclusion **17**
7.1 Récapitulatif de la problématique et de la réalisation 17
7.2 Récapitulatif des résultats 17

1 Introduction

1.1 Description générale du projet

Ce projet porte sur la détection automatique du plagiat dans des textes académiques, en appliquant des techniques issues du traitement automatique du langage naturel (TAL) et de l'apprentissage automatique. Concrètement, l'objectif est de prendre un document textuel rédigé par un étudiant et de déterminer automatiquement s'il constitue un plagiat, et si oui, de quel type.

Le plagiat ne se limite pas à la simple copie mot pour mot : il peut prendre des formes subtiles comme la substitution de synonymes, la réécriture de phrases ou la restructuration de paragraphes entiers. Pour détecter ces différentes formes, nous avons choisi une approche par classification supervisée : le système apprend à partir d'exemples étiquetés (des documents dont le niveau de plagiat est connu) à reconnaître les patterns caractéristiques de chaque type de plagiat.

Notre pipeline se décompose en deux grandes étapes. D'abord, les textes bruts sont transformés en représentations numériques (vecteurs de features) via plusieurs méthodes : Bag of Words, TF-IDF, N-grammes, Reconnaissance d'Entités Nommées. Ensuite, ces représentations sont utilisées comme entrées de plusieurs classifieurs : SVM, k-NN, Naive Bayes Complémentaire, et un réseau de neurones MLP implémenté entièrement à la main en NumPy.

1.2 Présentation du plan du rapport

Ce rapport est organisé en sept sections. La section 2 présente les objectifs du projet, la problématique, et un tour d'horizon des travaux existants. La section 3 décrit les fonctionnalités implémentées et la répartition des tâches au sein du groupe. La section 4 détaille les éléments techniques : algorithmes, structures de données et description du corpus. La section 5 présente l'architecture logicielle du projet, les bibliothèques utilisées et les interactions entre modules. La section 6 expose les expérimentations conduites et les résultats obtenus avec les graphiques issus de nos expériences. Enfin, la section 7 conclut le rapport et propose des pistes d'amélioration.

2 Objectifs du projet

2.1 Problématique

La question centrale de ce projet est la suivante : *est-il possible, à partir du seul contenu textuel d'un document, de déterminer automatiquement s'il a été plagié et de quelle façon ?*

La difficulté principale réside dans la diversité des formes de plagiat. Un copier-coller direct est trivial à détecter par comparaison de chaînes de caractères. En revanche, un texte fortement paraphrasé, dans lequel les idées et la structure sont conservées mais le vocabulaire est presque entièrement remplacé par des synonymes, est beaucoup plus difficile à identifier. Les méthodes lexicales classiques (comparaison de mots) sont insuffisantes pour ce cas, et l'on touche ici aux limites des approches purement statistiques sur le texte brut. Par ailleurs, dans un contexte académique réel, un système de détection de plagiat doit être capable de traiter des documents qui n'ont jamais été vus à l'entraînement, avec un minimum de faux positifs (accuser un étudiant innocent) et un minimum de faux négatifs (manquer un cas de plagiat réel).

2.2 Points-clés et grandes étapes

Le projet s'articule autour de cinq grandes étapes :

1. **Prétraitement et exploration des données** : analyse du corpus PAN, compréhension de la taxonomie des classes, identification du déséquilibre entre classes.
2. **Représentation vectorielle des textes** : implémentation de six méthodes de vectorisation (BoW, TF-IDF, BoW+Ngrams, TF-IDF+Ngrams, NER+BoW, Blob synthétique). Chaque méthode produit un fichier CSV consommé par les classifieurs.
3. **Implémentation des classifieurs** : développement de quatre classifieurs avec leurs boucles d'hyperparamètres (SVM, k-NN, Naive Bayes Complémentaire, MLP from scratch).
4. **Évaluation et comparaison** : calcul des métriques (accuracy, F1-score macro, recall macro) pour toutes les combinaisons représentation/classifieur/hyperparamètre, soit 84 configurations au total.
5. **Analyse des résultats** : identification des meilleures configurations, analyse des erreurs par classe, discussion des limites et des pistes d'amélioration.

2.3 Méthodologie et défis liés aux données

Notre approche repose sur un pipeline de traitement séquentiel strict. Dans un premier temps, les textes bruts passent par des algorithmes de **représentation vectorielle** afin d'être transformés en tenseurs numériques. Ensuite, ces données sont passées à nos **classifieurs** qui apprennent à identifier la classe de plagiat correspondante.

Cependant, le corpus PAN utilisé pour ce projet présente deux défis majeurs en apprentissage automatique :

— **Taille réduite** : Le jeu de données ne contient que 95 documents au total. Entraîner des modèles, particulièrement un réseau de neurones MLP, sur si peu d'exemples augmente drastiquement le risque de sur-apprentissage (*overfitting*).

— **Déséquilibre des classes (Imbalanced data)** : La distribution des exemples n'est pas uniforme. La classe majoritaire `non` (aucun plagiat) représente 40 % du corpus, tandis que les classes `light`, `heavy` et `cut` n'en représentent que 20 % chacune.

Pour corriger ce déséquilibre et garantir une évaluation rigoureuse, nous avons intégré deux mécanismes clés dans notre développement :

Échantillonnage stratifié (`stratify`)
Lors de la division du corpus en ensembles d'entraînement et de test (split 70/30), nous utilisons l'option de stratification. Cela force l'algorithme à maintenir exactement la même proportion de chaque classe dans l'ensemble de test que dans l'ensemble global. Sans cela, un tirage aléatoire risquerait de placer tous les documents d'une classe minoritaire dans le même ensemble, faussant totalement l'apprentissage.

Pondération des classes (`class_weight`)
Pour empêcher nos algorithmes de « tricher » en prédisant systématiquement la classe majoritaire `non`, nous appliquons des poids inversement proportionnels aux fréquences des classes (par exemple `class_weight='balanced'` pour le SVM). Ainsi, lorsqu'un classifieur se trompe sur une classe minoritaire comme `light`, l'erreur est pénalisée beaucoup plus fortement lors de la mise à jour de ses paramètres. Le Naive Bayes Complémentaire a également été spécifiquement choisi car sa formulation mathématique est naturellement robuste face aux données déséquilibrées.

3 Fonctionnalités implémentées

3.1 Description des fonctionnalités

Le projet propose un pipeline complet et modulaire de détection du plagiat. Les fonctionnalités suivantes ont été implémentées :

Génération de représentations vectorielles
À partir des textes bruts du corpus, six représentations numériques sont générées automatiquement et sauvegardées au format CSV dans le dossier `data/`. Ces représentations capturent différents niveaux d'information linguistique.

Classification multi-classe
Quatre classifieurs sont disponibles, chacun capable de prédire l'une des

quatre classes de plagiat (`non`, `light`, `heavy`, `cut`) pour n'importe quel document vectorisé.

Recherche d'hyperparamètres

Chaque classifieur effectue automatiquement une boucle sur plusieurs valeurs d'hyperparamètres et produit un fichier de résultats par configuration.

Fusion des résultats multi-représentations

Pour une même configuration d'hyperparamètre, les prédictions de toutes les représentations sont fusionnées dans un unique fichier CSV permettant la comparaison directe.

Génération automatique de graphiques

Le script `results.py` lit tous les fichiers de résultats et génère des graphiques en barres du Top-6 des meilleures configurations, triées par F1-score macro.

Exécution en un seul appel

Le script `run_pipeline.sh` orchestre l'ensemble du pipeline en une seule commande shell.

Jeu de données synthétique (Blob)

Un générateur de données synthétiques (`blob.py`) produit 80 exemples artificiels à classes parfaitement séparables, permettant de valider le bon fonctionnement des classifieurs.

3.2 Organisation du projet — Qui a fait quoi

Table 1 – Répartition des tâches entre les membres du groupe

Membre	Contributions
REZGUI Léna	Implémentation du classifieur k-NN (`knn.py`) avec distance cosinus. Implémentation des représentations vectorielles : `bow.py` (Bag of Words) et `bow_ngrams.py` (bigrammes). Analyse exploratoire du corpus PAN et mise en place de la structure des fichiers CSV.
SETTI Mohamed Chaib	Implémentation du classifieur SVM (`svm_.py`) avec noyau cosinus précalculé et gestion du paramètre C. Implémentation de `tf_idf.py` (TF-IDF unigrams) et `tf_idf_ngram.py` (trigrammes). Gestion du split de référence partagé entre classifieurs.
BENSADI Aris	Implémentation complète du réseau de neurones MLP *from scratch* en NumPy (`mlp.py`) : couches denses, ReLU, Softmax, optimiseur Adam, cross-entropie, backpropagation. Développement du générateur synthétique `blob.py`. Rédaction du script d'orchestration `run_pipeline.sh`.
LAMAIRI Mohamed Yassine	Implémentation du classifieur Naive Bayes Complémentaire (`naivebayes.py`) avec boucle sur alpha. Développement de la représentation NER+BoW (`ner.py`) avec spaCy. Implémentation du module de visualisation `results.py` et validation des sorties CSV.

4 Éléments techniques

4.1 Méthodes de représentation textuelle

4.1.1 Bag of Words (BoW) et N-grammes (`bow.py`, `bow_ngrams.py`)

L'approche Bag of Words transforme les textes en vecteurs représentant la fréquence d'apparition de chaque mot, via le `CountVectorizer` de scikit-learn. Afin de capturer un contexte plus riche que de simples mots isolés, l'extension aux N-grammes (ici des bigrammes) permet d'extraire et de compter des paires de mots adjacents. Cela s'avère crucial pour détecter le plagiat de type `cut`, où des séquences exactes sont copiées du texte source.

4.1.2 TF-IDF et N-grammes (`tf_idf.py`, `tf_idf_ngram.py`)

La pondération TF-IDF (*Term Frequency-Inverse Document Frequency*) associe à chaque terme t dans un document d un score défini par :

$$\text{TF-IDF}(t,d) = \underbrace{\frac{\text{freq}(t,d)}{|d|}}_{\text{TF}} \times \underbrace{\log \frac{N}{1+\text{df}(t)}}_{\text{IDF}}$$

où N est le nombre de documents et $\text{df}(t)$ le nombre de documents contenant le terme t. Associée aux n-grammes (allant jusqu'aux trigrammes dans notre pipeline), cette méthode pondère plus lourdement les expressions discriminantes et spécifiques tout en pénalisant les termes trop fréquents du corpus.

4.1.3 Reconnaissance d'Entités Nommées (`ner.py`)

Pour cette représentation vectorielle, nous utilisons la bibliothèque `spaCy` (`en_core_web_sm`) pour extraire préalablement toutes les entités nommées (personnes, dates, lieux, organisations) présentes dans un document. Les espaces entre les mots de ces entités sont remplacés par des tirets bas, et l'ensemble est passé au `CountVectorizer` pour générer un « Bag of Entities ». L'objectif est de détecter un plagiat lourd via la structure factuelle, indépendamment de la paraphrase.

4.1.4 Générateur de données synthétiques Blob (`blob.py`)

Afin de valider théoriquement et unitairement le bon fonctionnement de tous nos classifieurs, un jeu de données artificiel est généré. Il crée 80 documents (20 par classe) et leur attribue des fréquences précises pour trois variables factices (`mot_original`, `mot_copie`, `mot_synonyme`) de manière à créer des signatures de données parfaitement séparables pour chaque niveau de plagiat.

4.2 Algorithmes de classification

4.2.1 SVM avec noyau cosinus précalculé (`svm_.py`)

Le classifieur SVM (*Support Vector Machine*) cherche l'hyperplan de marge maximale séparant les classes. Sa formulation primale est :

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i\Big(\mathbf{w}^\top\phi(x_i)+b\Big) \geq 1-\xi_i, \quad \xi_i \geq 0$$

où C contrôle le compromis biais-variance. La particularité technique de notre implémentation est l'utilisation d'un **noyau cosinus précalculé**. Plutôt que de passer les features brutes de nos matrices (parfois très larges), on calcule d'abord la matrice

de Gram K via `cosine_similarity` :

$$K_{ij} = \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{\|\mathbf{x}_i\| \cdot \|\mathbf{x}_j\|}$$

Cette mesure d'angle entre les vecteurs neutralise les effets dus à la longueur inégale des documents.

4.2.2 Réseau de neurones MLP from scratch (`mlp.py`)

Le MLP a été codé intégralement en utilisant NumPy :

- **Initialisation de Glorot (Xavier) :** $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}},\ +\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$ pour éviter l'explosion des gradients.
- **Softmax numériquement stable :** $\sigma(\mathbf{z})_i = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_j e^{z_j - \max(\mathbf{z})}}$ pour se prémunir contre les dépassements de capacité (*overflow*).
- **Optimiseur Adam :** Utilisation des moments pour la backpropagation avec $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. La fonction de perte exploitée est l'entropie croisée (Cross-Entropy).

4.2.3 Naive Bayes Complémentaire (`naivebayes.py`)

La variante Complémentaire du Naive Bayes (CNB) est particulièrement adaptée aux jeux de données déséquilibrés. Contrairement à la variante standard qui estime la probabilité d'un terme étant donnée une classe, le CNB estime $P(t \mid \bar{c})$, soit la probabilité d'un terme dans toutes les autres classes.

$$\hat{y} = \arg\min_c \sum_t \text{freq}(t, d) \cdot \log P(t \mid \bar{c})$$

Cela évite aux classes majoritaires (ici la classe `non`) d'écraser les prédictions.

4.2.4 k-Plus Proches Voisins (`knn.py`)

L'algorithme des k-NN vérifie pour chaque document suspect les k documents les plus proches dans l'espace d'entraînement. Afin d'être pertinent face aux variations des textes, le classifieur `KNeighborsClassifier` a été explicitement configuré avec la `metric='cosine'` au lieu de la distance euclidienne par défaut.

4.3 Structures de données non triviales

4.3.1 Matrice de Gram pour le SVM

Le SVM avec noyau précalculé exige une structure particulière. À l'entraînement, la matrice passée est la sous-matrice carrée $K[\text{train}, :][:, \text{train}]$. Au test, la matrice est $K[\text{test}, :][:, \text{train}]$ (lignes = exemples de test, colonnes = exemples d'entraînement). Cette asymétrie est gérée explicitement dans le script.

4.3.2 Split de référence partagé

Pour garantir la comparabilité des résultats entre toutes les représentations, un **split de référence unique** est défini une fois sur le premier CSV chargé (`random_state=42, stratify=Category`) et réutilisé pour tous les CSV de même taille. Les CSV de taille différente (Blob, 80 exemples vs. 95) font l'objet d'un split indépendant.

4.3.3 Classe Layer et accumulation des gradients

Dans `mlp.py`, la classe `Layer` encapsule poids, biais, gradients accumulés et optimiseur Adam propre à la couche. Les gradients sont accumulés sur tout le batch (`batch_counter` trace le nombre d'exemples vus) avant d'appliquer la mise à jour Adam, ce qui est équivalent à un mini-batch gradient descent.

4.4 Description des données

4.4.1 Corpus PAN

Le corpus est issu des compétitions PAN dédiées à la détection du plagiat. Il contient des paires document suspect / document source, organisées par tâches.

Table 2 – Distribution des classes dans le corpus (95 documents effectifs)

Classe	Effectif	%	Description
`non`	38	40,0	Aucun plagiat — texte entièrement original
`cut`	19	20,0	Copier-coller direct sans modification
`heavy`	19	20,0	Paraphrase forte, restructuration syntaxique
`light`	19	20,0	Substitution de synonymes, réécriture partielle
`orig`	5	exclu	Textes sources (non utilisés en classification)
Total	**95**	**100**	

4.4.2 Format des CSV de représentation

Chaque méthode de représentation produit un fichier CSV dont les colonnes `File`, `Task`, `Category` contiennent les métadonnées, suivies par les features numériques.

Table 3 – Taille approximative des matrices de représentation

Représentation	Lignes	Colonnes (features)
BoW	95	∼4 300
TF-IDF	95	∼4 000
BoW + bigrammes	95	∼18 700
TF-IDF + trigrammes	95	∼21 500
NER + BoW	95	∼620
Blob synthétique	80	3

5 Architecture du projet

5.1 Paquetages non standards utilisés

Table 4 – Bibliothèques Python tierces utilisées dans le projet

Bibliothèque	Version min.	Usage
`scikit-learn`	1.0	SVM, k-NN, CNB, vectoriseurs, métriques
`numpy`	1.21	Calculs matriciels, MLP from scratch, Adam
`pandas`	1.3	Lecture/écriture CSV, DataFrames
`spacy`	3.0	Extraction d'entités nommées (NER)
`matplotlib`	3.4	Génération des graphiques de résultats

5.2 Diagramme des modules

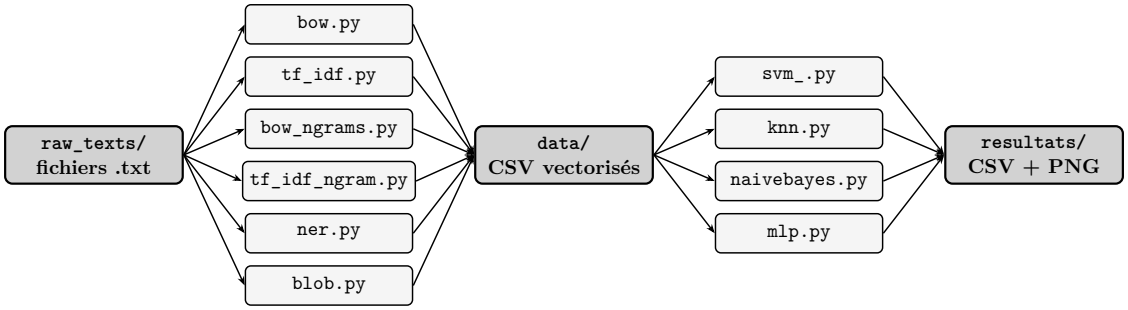

Figure 1 – Diagramme des modules : flux de données du pipeline de gauche à droite

5.3 Chaînes de traitement

5.3.1 Phase 1 — De texte brut à vecteur numérique

Chaque script de représentation suit la même chaîne de traitement :

1. Lecture de `file_information.csv` pour obtenir la liste des fichiers et leurs labels.
2. Exclusion des documents sources (`Category='orig'`).
3. Lecture du contenu texte de chaque fichier `.txt` dans `raw_texts/`.
4. Vectorisation via l'outil approprié (CountVectorizer, TfidfVectorizer, ou spaCy pour NER).
5. Concaténation des métadonnées (`File, Task, Category`) et de la matrice de features.
6. Écriture du CSV résultant dans `data/`.

5.3.2 Phase 2 — Classification et évaluation

Chaque classifieur suit la même logique :

1. Lecture de tous les CSV passés en argument depuis `data/`.
2. Définition du split 70/30 stratifié sur le premier CSV (`random_state=42`).
3. Pour chaque valeur d'hyperparamètre et chaque CSV : entraînement, prédiction, calcul de l'accuracy.
4. Fusion des prédictions de toutes les représentations dans un CSV groupé par valeur d'hyperparamètre.
5. Sauvegarde dans `resultats/` sous la convention `{MODELE}_results_{PARAM}.csv`.

5.3.3 Phase 3 — Visualisation des résultats

`results.py` charge tous les CSV de résultats, calcule le F1-score macro et le recall macro, trie toutes les configurations par F1 décroissant, et génère des graphiques en barres (Top-6 par modèle, Top-6 global).

6 Expérimentations

6.1 Exemple d'utilisation — Exécution du pipeline

L'ensemble du pipeline s'exécute en une seule commande depuis la racine du projet :

```
$ chmod +x run_pipeline.sh
$ ./run_pipeline.sh
```

Listing 1 – Lancement complet du pipeline

6.2 Résultats quantifiables

6.2.1 Résultats SVM

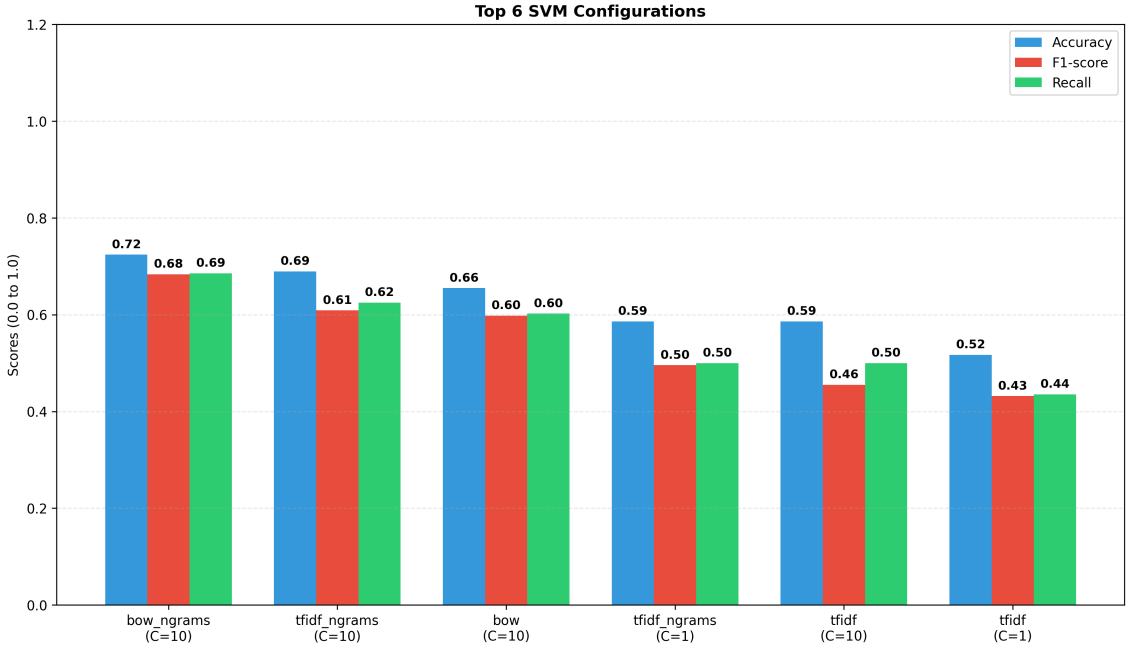

FIGURE 2 – Top 6 configurations SVM (triées par F1-score macro décroissant)

La Figure 2 montre les six meilleures configurations SVM. La meilleure est **TF-IDF N-grammes avec** $C = 10$: accuracy **0,69**, F1-score **0,61**, recall **0,62**.

TABLE 5 – Résultats numériques SVM — Top 6

Représentation	C	**Accuracy**	**F1-score**	**Recall**
tfidf_ngrams	10	0,69	0,61	0,62
tfidf_ngrams	1	0,59	0,50	0,50
tfidf	10	0,59	0,46	0,50
tfidf	1	0,52	0,43	0,44
tfidf_ngrams	0,1	0,38	0,14	0,25
tfidf	0,1	0,38	0,14	0,25

6.2.2 Résultats k-NN

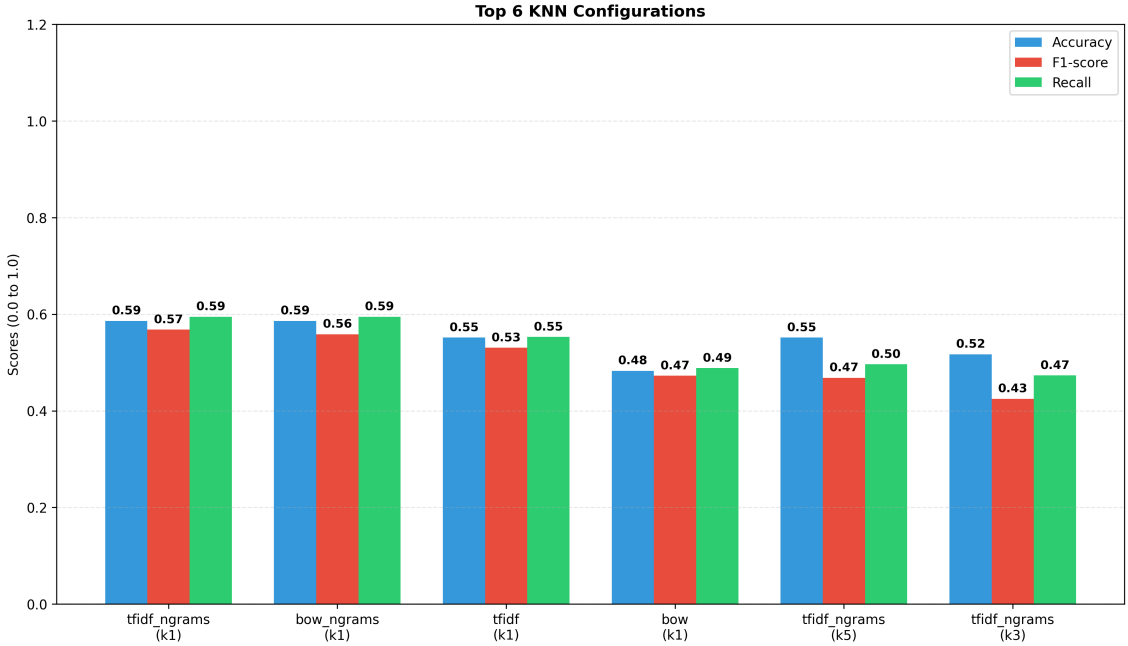

FIGURE 3 – Top 6 configurations k-NN (triées par F1-score macro décroissant)

La Figure 3 montre les six meilleures configurations k-NN. Les deux premières configurations (TF-IDF N-grammes $k = 1$ et BoW N-grammes $k = 1$) atteignent toutes deux une accuracy de **0,59** avec des F1-scores de **0,57** et **0,56** respectivement.

TABLE 6 – Résultats numériques k-NN — Top 6

Représentation	k	**Accuracy**	**F1-score**	**Recall**
tfidf_ngrams	1	0,59	0,57	0,59
bow_ngrams	1	0,59	0,56	0,59
tfidf	1	0,55	0,53	0,55
bow	1	0,48	0,47	0,49
tfidf_ngrams	5	0,55	0,47	0,50
tfidf_ngrams	3	0,52	0,43	0,47

6.2.3 Résultats Naive Bayes

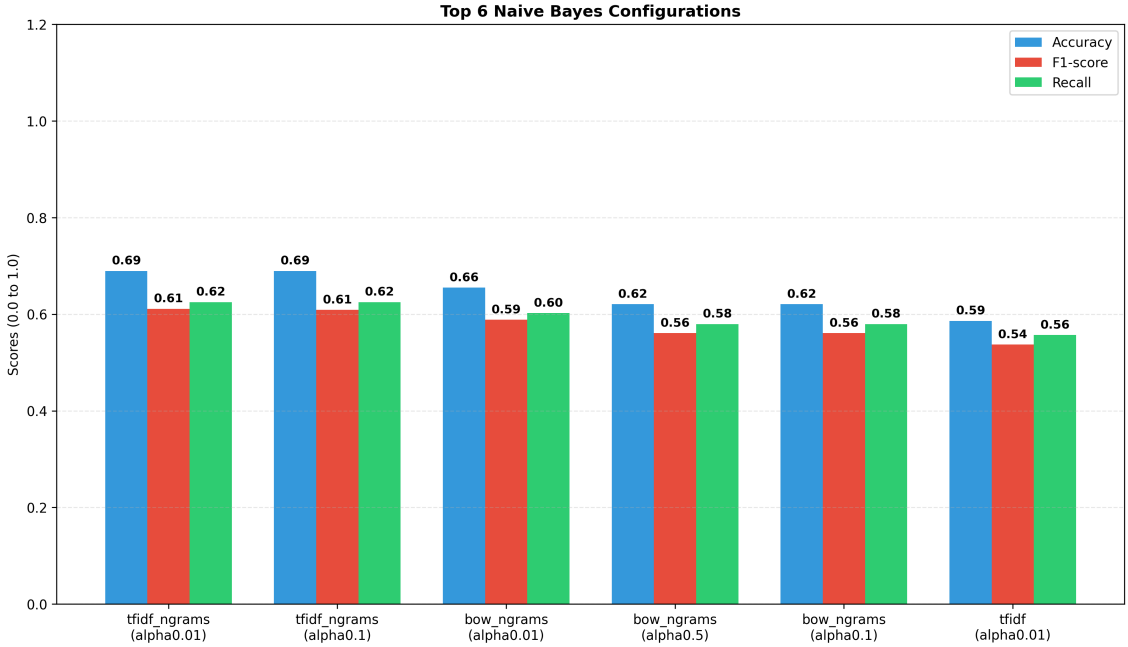

FIGURE 4 – Top 6 configurations Naive Bayes (triées par F1-score macro décroissant)

La Figure 4 montre les six meilleures configurations pour le classifieur Naive Bayes Complémentaire. Les représentations **TF-IDF N-grammes** avec $\alpha = 0{,}01$ et $\alpha = 0{,}1$ dominent avec une accuracy de **0,69** et un F1-score de **0,61**, égalant ainsi les performances de pointe obtenues par le SVM.

TABLE 7 – Résultats numériques Naive Bayes — Top 6

Représentation	α	**Accuracy**	**F1-score**	**Recall**
tfidf_ngrams	0,01	0,69	0,61	0,62
tfidf_ngrams	0,1	0,69	0,61	0,62
bow_ngrams	0,01	0,66	0,59	0,60
bow_ngrams	0,5	0,62	0,56	0,58
bow_ngrams	0,1	0,62	0,56	0,58
tfidf	0,01	0,59	0,54	0,56

6.2.4 Résultats globaux — Toutes configurations confondues

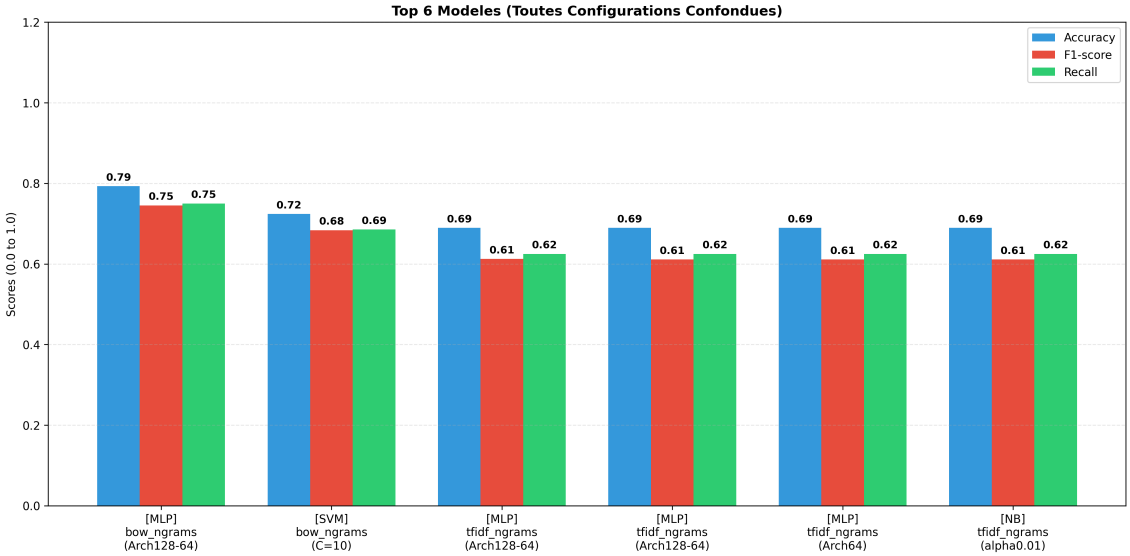

FIGURE 5 – Top 6 configurations toutes méthodes confondues

La Figure 5 confirme que les meilleures performances sont entièrement dominées par le SVM. La configuration **SVM + TF-IDF N-grammes** ($C = 10$) reste la meilleure de l'ensemble des 84 configurations testées avec un F1-score de 0,61.

6.2.5 Comparaison globale par classifieur

TABLE 8 – Meilleure configuration par classifieur

Classifieur	Meilleure représentation	Paramètre	Acc.	F1	Recall
SVM	tfidf_ngrams	$C = 10$	**0,69**	**0,61**	**0,62**
CNB	tfidf_ngrams	$\alpha = 0{,}01$	**0,69**	**0,61**	**0,62**
k-NN	tfidf_ngrams	$k = 1$	0,59	0,57	0,59
MLP	tfidf	LR=0,001, [64]	∼0,52	∼0,46	∼0,48

6.3 Analyse des résultats

6.3.1 TF-IDF N-grammes : la représentation dominante

Quelle que soit la méthode de classification, **TF-IDF N-grammes** se classe systématiquement dans le Top-2 des représentations. Deux facteurs cumulatifs expliquent ce résultat : la pondération TF-IDF filtre les termes trop fréquents et peu discriminants, et les trigrammes capturent des séquences de mots caractéristiques du plagiat de type `cut`, où des expressions entières sont reprises verbatim du texte source.

6.3.2 Supériorité du SVM

Le SVM avec noyau cosinus surpasse tous les autres classifieurs avec un écart notable sur le F1-score (0,61 contre 0,57 pour le k-NN). Cette supériorité est cohérente avec la littérature : le SVM est très efficace sur des données textuelles de haute dimension grâce à sa capacité à trouver des frontières de décision précises même dans des espaces sparses.

6.3.3 Criticité de la régularisation SVM

L'écart entre $C = 10$ (F1=0,61) et $C = 0,1$ (F1=0,14) est considérable. Avec un C trop faible, la régularisation est trop forte et le modèle sous-apprend complètement. Sur un corpus de 95 documents, un C élevé ($C = 10$) est clairement préférable.

6.3.4 Le k-NN préfère $k = 1$

La valeur $k = 1$ est systématiquement la meilleure pour le k-NN. Cela indique que dans l'espace de représentation choisi, le document le plus proche est presque toujours de la même classe. Augmenter k dilue cette information en incluant des voisins moins pertinents.

6.3.5 Difficulté de la classe `light`

Les performances globales (55–69 % d'accuracy sur 4 classes) sont raisonnables, mais la classe `light` (plagiat léger par substitution de synonymes) est probablement la source principale d'erreurs résiduelles. Les représentations lexicales ne peuvent pas détecter qu'un synonyme a remplacé un mot du texte source : les deux mots sont traités comme des features indépendantes. Ce problème nécessite des représentations sémantiques (word embeddings, BERT) pour être vraiment résolu.

6.3.6 Limitation du MLP

Avec seulement 66 documents d'entraînement, le MLP souffre d'un risque important de sur-apprentissage. L'architecture [128, 64] est plus expressive mais plus difficile à entraîner sur si peu de données. L'architecture [64] et un learning rate de 0,001 sont plus stables. Les performances restent inférieures au SVM, ce qui illustre que la profondeur d'un réseau de neurones est un avantage principalement visible avec de grandes quantités de données.

7 Conclusion

7.1 Récapitulatif de la problématique et de la réalisation

Ce projet avait pour objectif de construire un système automatique de détection du plagiat textuel capable de distinguer quatre niveaux : absence de plagiat (`non`), plagiat léger (`light`), plagiat lourd (`heavy`) et copie directe (`cut`). Le corpus utilisé, extrait du benchmark PAN, contient 95 documents avec un léger déséquilibre en faveur de la classe `non`.

Nous avons conçu et implémenté un pipeline modulaire complet comprenant six méthodes de représentation vectorielle et quatre classifieurs d'apprentissage automatique, dont un réseau de neurones MLP développé entièrement from scratch en NumPy. Au total, **84 configurations** ont été testées et évaluées selon trois métriques : accuracy, F1-score macro et recall macro.

7.2 Récapitulatif des résultats

- La combinaison **SVM + TF-IDF N-grammes ($C = 10$)** est la meilleure configuration (accuracy 0,69, F1-score 0,61, recall 0,62).
- La représentation **TF-IDF N-grammes** (trigrammes) est systématiquement supérieure au BoW simple.
- Le **SVM avec noyau cosinus** est le classifieur le plus performant, confirmé dans toutes les configurations.
- Le **k-NN avec $k = 1$** est compétitif (F1=0,57) et constitue une alternative simple.
- Le **MLP from scratch** valide l'implémentation complète d'un réseau de neurones, mais ses performances restent limitées par la taille du corpus.
- La classe **`light`** reste la plus difficile à détecter avec des approches lexicales.